Printer Job Management System (FIFO Queue)

Course Code: CSS2281 **Course Name:** C Programming and Data Structures Lab **Activity:** Problem-Based Learning (PBL) Activity **Submission Date:** 28th October 2026

👥 Group Details

Group Number: [Insert Group Number Here]

- **Member 1:** [AKSHAY M] - [20251ECE0077]
- **Member 2:** [PUNITH S] - [20251ECE0076]
- **Member 3:** [DAYANANDA N] - [20251ECE0078]
- **Member 4:** [NANDYALA TARAKESHWAR] - [20251ECE0079]
- **Member 5:** [KAKANOORU AJAY REDDY] - [20251ECE0080]

📌 Project Overview

This project implements a **Printer Job Management System** using a First-In-First-Out (FIFO) **Queue** data structure. In real-world operating systems and print servers, when multiple users submit documents to a printer, the requests are held in a "print spooler" (queue). This program simulates that behavior, ensuring that documents are printed in the exact order they were received.

📝 Step-by-Step Code Processing

Here is the step-by-step execution flow of the C program:

Step 1: Initialization

- The program starts in the `main()` function by creating a `PrintQueue` structure called `spooler`.
- It calls `initQueue(&spooler)` to set up the empty queue. It sets `front = 0`, `rear = -1`, and `count = 0` (meaning there are currently 0 jobs waiting).

Step 2: The Main Menu Loop

- A continuous `while(1)` loop starts, displaying the main menu to the user: Submit Job, Process Job, View Status, or Exit.
- The program waits for the user's `switch-case` choice.

Step 3: Submitting a Job (Enqueue Operation)

- If the user selects **1**, the program asks for document details (name, pages, username).

- It calls the `addJob()` function.
- *Processing:* First, it checks `isFull()`. If `count == MAX`, it rejects the job to prevent overflow. Otherwise, it moves the `rear` index forward using circular logic `(rear + 1) % MAX`, stores the new job details at that position, and increases the `count` by 1.

Step 4: Processing a Job (Dequeue Operation)

- If the user selects **2**, it calls the `processJob()` function.
- *Processing:* First, it checks `isEmpty()`. If `count == 0`, it alerts the user that the printer is idle. Otherwise, it retrieves the job currently sitting at the `front` index and prints its details to the screen.
- It then moves the `front` index forward `(front + 1) % MAX` (effectively removing the job from the queue) and decreases the `count` by 1.

Step 5: Viewing the Queue

- If the user selects **3**, it calls `displayQueue()`.
- *Processing:* It uses a `for` loop that runs exactly `count` times. It calculates the correct index starting from `front` and prints the details of every job currently waiting in line.

Step 6: Exit

- If the user selects **4**, the program calls `exit(0)` and safely terminates the application.

⚙️ Features

1. **Submit Print Job (Enqueue):** Add a new document to the printer spooler with details like Document Name, Number of Pages, and User Name.
2. **Process Next Job (Dequeue):** Simulates the printer printing the document at the front of the queue.
3. **View Queue Status:** Displays all pending print jobs currently waiting in the spooler.
4. **Boundary Handling:** Includes robust checks for Queue Overflow (printer spooler full) and Queue Underflow (no jobs to print).

🚀 How to Run the Program

1. Open a terminal or command prompt in your project folder.
2. Compile the program using a C/C++ compiler:

```shell
g++ "printer job management.cpp" -o printer_queue
```

3. Execute the compiled program:
    - On Windows: `printer_queue.exe`

    - On Mac/Linux: `./printer_queue`
4. Follow the interactive menu on the screen to add and process print jobs.

🛠️ Data Structures Used

- **Struct:** To store individual print job details (`job_id`, `document_name`, `pages`, `user_name`).
- **Circular Queue (Array-based):** To manage the print jobs efficiently without needing to shift elements in memory after a job is processed.